

스크린

강사

정준모

조용민

김민규

하지훈

한동훈

차재민

최현서

차희성

김정훈

박준재

방현수

김정찬

장원우

김형규

김현수

김진실

김미성

최재희

김가은

정석환

이주연

박재홍

배성우

임태현

최민호

이서범

오희진

보조강사

최인영

조재경

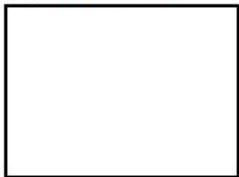
주동민

7.20.(월)

가상환경

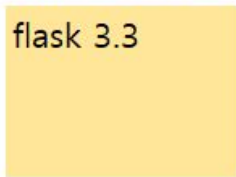
PC

Python 3.14



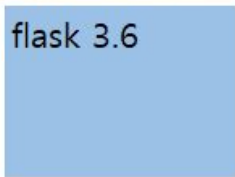
가상환경 1 - a

flask 3.3



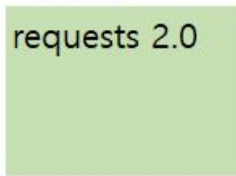
가상환경 2 - b

flask 3.6



가상환경 3 - .venv

requests 2.0



010-8478-8182 김남현

seorab@kakao.com

파이썬 설치 후 해야 할 작업

1. 가상환경 생성
2. 가상환경 활성화
3. 사용할 도구 설치 (pip install 도구명)
 - pip install ipykernel

개발 환경 설정

1. 파이썬 설치

2. VS Code 설치 + Python 확장 프로그램 설치

3. 가상환경 생성

→ `python -m venv` 가상환경명

ex) `python -m venv .venv`

4. 가상환경 활성화

→ `call .venv/Scripts/Activate`

주피터 노트북 파일 생성

1. (VS Code 에서)

명령 팔레트(Shift + Ctrl + P)에서 create jupyter 검색

2. 파일 저장

(저장하지 않으면 파일이 생성되지 않음)

네이밍 컨벤션 Naming Convention

1. 카멜 camel case : person age → personAge
2. 파스칼 pascal case : person age → PersonAge
3. 스네이크 snake case : person age → person_age
4. 케밥 kebab case : person age → person-age

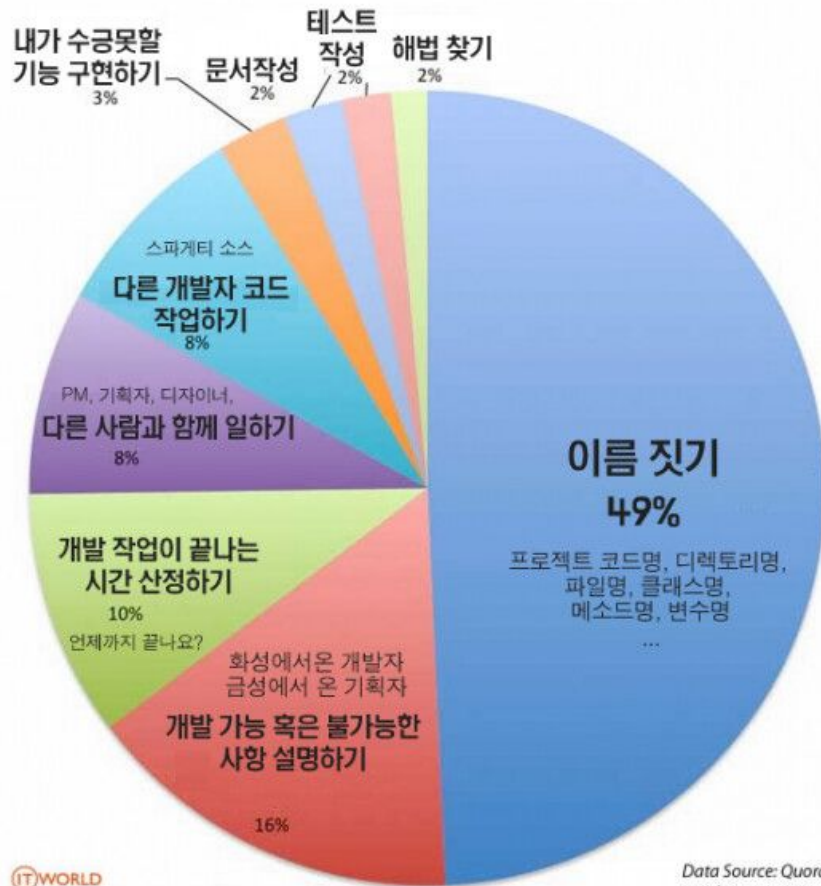
변수

값을 저장하기 위한 공간

복잡하고 긴 값을 미리 기억해놓고 재사용

기존의 값을 계속 활용

프로그래머가 가장 힘들어하는 일은?



변수명 짓기

변수명 짓기

<https://www.curiousstore.com/#!/util/naming>

숫자 자료형

정수, 실수, 2/8/16진수

1. 사칙연산(+ - * /), 몫(`//`), 나머지(`%`), 지수(`**`)

2. 다른 형태의 자료를 숫자로 변환

- 정수로 변환 → `int(다른 형태의 자료)`

- 실수로 변환 → `float(다른 형태의 자료)`

연습문제

초를 '00시 00분 00초' 형태로 변환하여 출력하는 프로그램을 만들려고 합니다.
몫 연산자(//)와 나머지 연산자(%), 그리고 f-string 포매팅을 활용하여
작성해보세요.

전체 시간 (3703초)

sec = 3703

print(f'{hour}시간 {minute}분 {second}초') → 1시간 1분 43초

페이징 공식

1. 페이지 이동을 위한 네비게이션 번호를 계산하는 식

1page → 1 ~ 10

7page → 1 ~ 10

16page → 11 ~ 20

첫번째 페이지 번호 : $(\text{현재 페이지 번호} - 1) // 10 * 10 + 1$

마지막 데이터 번호 : 첫번째 페이지 번호 + 9

페이징 공식

2. 조회할 데이터 번호를 계산하는 식

1page → 1 ~ 10

2page → 11 ~ 20

10page → 91 ~ 100

마지막 데이터 번호 : 현재 페이지 번호 * 10

첫번째 데이터 번호 : 마지막 데이터 번호 - 9

문자 자료형

작은 따옴표 또는 큰 따옴표 사용 **ex)** ' " '""

따옴표 1개 또는 3개

1. Indexing (해당 위치의 문자 1개 조회 - 문자 한개 추출)
 2. Slicing (지정된 범위의 문자 N개 조회 - 문자 여러개 잘라내기)
 3. Formatting (문자열의 틀(템플릿)을 잡고 가변적인 값만 {} 안에 쏙 꽂아 재사용)
3. 문자 관련 함수

문자 자료형 연습문제

<http://ggoreb.com/quiz/소나기.txt>

- "소나기" 개수 확인 → 2개

- "첩" 으로 시작하는 문장 찾기 → 첩덩굴이 엉키어 꽃을 달고 있었다.

7.21.(화)

암호화 문자열

t = "g fmnc wms bgblr rpylqjyrc gr zw fylb. rfyrq ufyr amknsrcpq ypc
dmp. bmgle gr gl zw fylb gq glcddgagclr ylb rfyr'q ufw rfgq rcvr gq qm
jmle. sqgle qrpgle.kyicrpylq() gq pcamkkclbcb. lmu ynnjw ml rfc spj."

before = 'abcdefghijklmnopqrstuvwxyz'

after = 'cdefghijklmnopqrstuvwxyzab'

개발 환경설정 정리

1. Python 설치

2. IDE Visual Studio Code

3. VSCode 확장프로그램 "Jupyter"

+ 가상환경 `python -m venv "가상환경명"`

+ 파이썬 도구 설치방법 `pip install "도구명"`

ex) `pip install ipykernel`

+ `ipynb(jupyter notebook file)` 생성

Select Kernel → 도구가 있는 환경 지정

문자 자료형 연습문제

<http://ggoreb.com/quiz/운수좋은날.txt>

- "김첨지" 개수 확인 → **49개**

김첨지 글자 사이에 띄어쓰기가 있는 경우도 있음

ex) 김 첨 지 김첨지 김 첨지

List

- N개의 자료를 하나의 자료로 묶어서 사용
- `list = []` 또는 `list = list()` 와 같이 사용
- 문자와 마찬가지로 indexing, slicing 기능 제공
- Dictionary와 함께 매우 자주 사용되는 자료형태

JSON 데이터 활용

1. requests 도구 설치

`pip install requests`

2. JSON 구조를 쉽게 파악

크롬 확장프로그램 `json formatter`

리스트 자료형 연습문제

<http://ggoreb.com/python/json/data4.jsp>

- "q" 데이터 추출하기 (indexing)

영화

API

[https://api.themoviedb.org/3/search/movie?api_key=cba95d401a14ab806ffc13a5052aab89&query=검색어](https://api.themoviedb.org/3/search/movie?api_key=<u>cba95d401a14ab806ffc13a5052aab89</u>&query=검색어)

[https://api.themoviedb.org/3/search/movie?api_key=eb5dea08ca14c5bdd7b9d451b555a0b2&query=검색어](https://api.themoviedb.org/3/search/movie?api_key=<u>eb5dea08ca14c5bdd7b9d451b555a0b2</u>&query=검색어)

(사용량 초과로 실행이 안되면) <http://ggoreb.com/api/movie.jsp>

포스터 이미지 경로

<https://image.tmdb.org/t/p/w185/qYczuua2tgjfxcdtLNDC0n4mOHZ.jpg>

Dictionary 자료형

중괄호 사용 ex) { 'a': 1, 'b': 2, 'c': 3 }

1. 리스트나 문자와 다르게 인덱스가 아닌 Key를 지정해서 조회
- dic['name'] 또는 dic.get('name')
2. 리스트와 문자에서 가능한 슬라이싱 기능 없음
3. Key가 존재하는지 확인하려면 in 사용
4. 딕셔너리 관련 함수 (keys, values, items, clear)

비교 연산자 우선순위 (and, or)

주소 = '부산'

자격증 = True

경력 = 2

자격증이 있거나 경력이 3년 이상이고 주소가 서울인 사람

자격증 == True or 경력 >= 3 and 주소 == '서울'

조건문

○ if - elif - else

1) 콜론 :

2) 들여쓰기

3) 조건식은 항상 True / False

4) 조건에 맞으면 if 아니면 elif

모든 조건에 맞지 아니면 else

→ 아래의 조건은 건너뛴 조건까지 포함

착시 / 착청

소리

<https://www.vocabulary.com/dictionary/laurel>

색상

[https://m.blog.naver.com/PostView.naver?isHttpsRedirect=true&blogId=tophoonn
&logNo=220285765406](https://m.blog.naver.com/PostView.naver?isHttpsRedirect=true&blogId=tophoonn&logNo=220285765406)

움직임

[https://postfiles.pstatic.net/20090711_172/panem_1247299241898qVE6y_gif/c1c
2b3fabfecb3fac5d7bdbac6ae_gbmaster_panem.gif?type=w2](https://postfiles.pstatic.net/20090711_172/panem_1247299241898qVE6y_gif/c1c2b3fabfecb3fac5d7bdbac6ae_gbmaster_panem.gif?type=w2)

조건문 연습문제) 평년 / 윤년

윤년인지 평년인지 구분하기

조건 1) 4로 나누어 떨어지는 해는 “윤년” (2004, 2016, ...)

조건 2) 100으로 나누어 떨어지는 해는 “평년” (1900, 2100, ...)

조건 3) 400으로 나누어 떨어지는 해는 “윤년” (1600, 2000, ...)

조건 4) 그 외 모든 해는 “평년” (1997, 2003, 2009, ...)

연습) 알람 시계 일찍 설정하기

조건 1) 24시간 표현으로 하루의 시작은 0:0 이고 끝은 23:59

조건 2) 시간을 입력할 때 시와 분을 띄어쓰기로 구분하고 불필요한 0은 사용하지 않음

ex) 10:10 → 10 10 / 00:10 → 0 10

조건 3) 입력된 시간에서 45분 앞당긴 결과 출력

ex) 14 45 → 14 00 / 23 40 → 22 55 / 0 45 → 0 0 / 0 30 → 23 45

```
value = input().strip().split()
```

```
h = int(value[0])
```

```
m = int(value[1])
```

반복문

○ while

- 1) 콜론 :
- 2) 들여쓰기
- 3) 조건식은 항상 True / False
- 4) 무한히 반복되지 않도록 조건식 주의

ex) while count < 5:

count += 1 # 반복할 때 마다 계속 증가시켜서 언젠가는 조건에 맞지
않도록

조건문 연습문제) 야구게임

숫자 3개를 입력해서 상황에 따라 스트라이크, 볼, 아웃을 판단하기

문제 설명) <http://ggoreb.com/quiz/baseball.html>

```
import requests as req  
res = req.get('http://ggoreb.com/quiz/baseball.txt')  
text = res.text  
정답 = [ text[0], text[1], text[2] ]  
입력 = [ '2', '1', '6' ]
```

7.22.(수)

조건문 연습문제) 야구게임 - 하드코딩, 답 직접 입력

```
import requests as req
res = req.get('http://ggoreb.com/quiz/baseball.txt')
text = res.text
정답 = [ text[0], text[1], text[2] ]
입력 = [ '1', '6', '2' ]
strike = 0 # 초기화
ball = 0
out = 0

if 정답[0] == 입력[0]: strike += 1
elif 정답[0] in 입력: ball += 1
else: out += 1

if 정답[1] == 입력[1]: strike += 1
elif 정답[1] in 입력: ball += 1
else: out += 1

if 정답[2] == 입력[2]: strike += 1
elif 정답[2] in 입력: ball += 1
else: out += 1

print(f'{strike}S {ball}B')
```

조건문 연습문제) 야구게임 - 반복문 적용

```
import requests as req
res = req.get('http://ggoreb.com/quiz/baseball.txt')
text = res.text
정답 = [ text[0], text[1], text[2] ]
입력 = [ '1', '6', '2' ]
strike = 0 # 초기화
ball = 0
out = 0
i = 0
while i <= 3:
    if 정답[i] == 입력[i]: strike += 1
    elif 정답[i] in 입력: ball += 1
    else: out += 1

print(f'{strike}S {ball}B')
```

조건문 연습문제) 야구게임 - 사용자 입력 + 답 입력 반복

```
import requests as req
res = req.get('http://ggoreb.com/quiz/baseball.txt')
text = res.text
정답 = [ text[0], text[1], text[2] ]
# 입력 = [ '1', '6', '2' ]
con = True
while con:
    입력 = list(input())
    strike = 0 # 초기화
    ball = 0
    out = 0
    i = 0
    while i < 3:
        if 정답[i] == 입력[i]: strike += 1
        elif 정답[i] in 입력: ball += 1
        else: out += 1
        i += 1
    if strike == 3:
        con = False
    print(f'{strike}S {ball}B')
```

가위바위보 게임 규칙 확인

```
com = '가위'  
player = '바위'
```

```
if com == player:  
    print('비겼습니다.')  
elif com == '가위':  
    if player == '바위':  
        print('플레이어 승리!')  
    else:  
        print('컴퓨터 승리!')  
elif com == '바위':  
    if player == '보':  
        print('플레이어 승리!')  
    else:  
        print('컴퓨터 승리!')
```

가위바위보 게임 규칙 확인

가위	바위	보	
0	1	2	
나 + 1 == 상대방 <=== 내가 졌음			
보 + 1 == 3			
(보 + 1) % 3 == 상대방			

가위바위보 게임 규칙 확인

가위0 바위1 보2

```
import random
```

```
상대방 = random.randint(0, 2)
```

```
나 = int(input())
```

```
if (나 + 1) % 3 == 상대방:
```

```
    print('상대방 승!')
```

```
elif 나 == 상대방:
```

```
    print('비김')
```

```
else:
```

```
    print('나 승!')
```

반복문 연습문제) 숫자 개수 확인하기

1 ~ 100 사이의 숫자 중 3을 포함하고 있는 숫자와 그 개수를 출력하기

3
13
23
30
31
32
33
34
35

37
38
39
43
53
63
73
83
93

총 개수:
19개

반복문

○ for

1) 콜론 :

2) 들여쓰기

3) 리스트

4) 반복횟수가 정해져 있음

로또 번호 생성하기 (6개, 중복될 수 있음)

요구사항) 중복된 번호가 들어가지 않도록 코드 수정하기

```
import random
```

```
로또번호 = list() # []
```

```
n = 0
```

```
while n < 6:
```

```
    n += 1
```

```
    로또번호.append( random.randint(1, 45) )
```

```
로또번호
```

로또 번호 생성하기

```
import random
로또번호 = list() # []
while len(로또번호) < 6:
    temp = random.randint(1, 45)
    if temp not in 로또번호:
        로또번호.append( temp )
로또번호
```

로또 번호 생성하기

```
import random
로또번호 = set()
while len(로또번호) < 6:
    temp = random.randint(1, 45)
    로또번호.add( temp )
로또번호
```

소나기 칩 반복 출력하기

```
import requests
```

```
url = 'http://ggoreb.com/python/file/소나기.txt'
```

```
res = requests.get(url)
```

```
res.encoding = 'utf8'
```

```
text = res.text
```

```
칩위치 = text.find('칩')
```

```
마침표위치 = text.find('.', 칩위치 + 1)
```

```
print(text[칩위치:마침표위치 + 1])
```

소나기 침 반복 출력하기 풀이

```
import requests
url = 'http://ggoreb.com/python/file/ 소나기.txt'
res = requests.get(url)
res.encoding = 'utf8'
text = res.text
```

```
마침표위치 = 0
```

```
while True:
    침위치 = text.find('침', 마침표위치)
    if 침위치 == -1: break
    마침표위치 = text.find('.', 침위치 + 1)
    print(text[침위치:마침표위치 + 1])
```

각 자리 숫자 합 출력하기

```
num = 215179  
total = 0
```

```
n1 = num % 10  
total += n1
```

```
n2 = (num // 10) % 10  
total += n2
```

```
n3 = (num // 100) % 10  
total += n3
```

```
n4 = (num // 1000) % 10  
total += n4
```

```
n5 = (num // 10000) % 10  
total += n5
```

```
n6 = (num // 100000) % 10  
total += n6
```

```
print(total)
```

각 자리 숫자 합 출력하기 풀이

```
num = 215179
```

```
total = 0
```

```
while True:
```

```
    n = num % 10
```

```
    total += n
```

```
    num //= 10
```

```
    if num == 0: break
```

```
print(total)
```

로또 1등 당첨 도전

1. 1등 번호 6자리 준비

2. 도전할 번호 입력

- 랜덤으로 6개 번호 생성

(너무 오래 걸릴수 있으므로 미리 3개는 입력!)

3. 1등 번호와 도전할 번호 비교

로또 1등 당첨 도전

```
import random
```

```
당첨번호 = [6, 11, 16, 19, 21, 32]
```

```
고정번호 = [6, 11, 16]
```

```
로또번호 = set(고정번호)
```

```
while len(로또번호) < 6:
```

```
    로또번호.add(random.randint(1, 45))
```

```
로또번호 = sorted(list(로또번호))
```

```
print(f'나의 로또번호: {로또번호}')
```

```
if 로또번호 == 당첨번호:
```

```
    print('성공!')
```

로또 확률 계산기

<https://parkminkyu.github.io/newLotto/lottoWin.html>

초코칩 단어 개수 세기 연습

1) 띄어쓰기 기준 각 단어를 분리

ex) "안촉촉한 초코칩 ..." → ["안촉촉한", "초코칩", ...]

2) 각 단어와 등장횟수 출력

ex) 안촉촉한 6

초코칩 6

나라에 2

초코칩이 4

...

```
import requests
url = 'http://ggoreb.com/python/file/chocochip.txt'
res = requests.get(url)
res.encoding = 'ms949'
text = res.text
```

초코칩 단어 개수 세기 연습

```
import requests

url = 'http://ggoreb.com/python/file/chocochip.txt'
# 초코칩      초코칩,
res = requests.get(url)
res.encoding = 'ms949'
text = res.text
words = text.split()
등장단어 = {}

for word in words:
    if word in 등장단어:
        등장단어[word] += 1
    else:
        등장단어[word] = 1

print(등장단어)
```

최대 공약수 구하기

$$a = 15, b = 6$$

- 1) a 와 b 중 적은 수 확인
- 2) 적은 수로 큰 수 나누기
- 3) 나머지가 0인 경우 최대 공약수 (반복 종료)
- 4) 현재 적은 수로 나누어지지 않으면 적은 수 - 1
- 5) 1이 될때까지 반복 후 종료

$$\text{최소 공배수} = (a * b) / \text{최대 공약수}$$

최대 공약수 구하기

```
a, b = 15, 6
for n in range(min(a, b), 0, -1):
    # print(n)
    if a % n == 0 and b % n == 0:
        print(n)
        break
```

최대 공약수 구하기 (파이썬의 math 도구 사용)

```
import math
```

```
a, b = 15, 6
```

```
최대공약수 = math.gcd(a, b)
```

```
최소공배수 = math.lcm(a, b)
```

```
print(최대공약수, 최소공배수)
```

리스트 + 딕셔너리 (반복문 적용)

미니 블로그 데이터를 아래와 같은 형식으로 출력하기

제목: 리액트에서 리스트 렌더링하기

댓글수: 5개

실제로 개발하다보면 `map` 함수를 진짜 많이 쓰는 것 같아요 😊

...

```
import requests
```

```
res = requests.get('http://ggoreb.com/api/mini-blog.jsp')
```

```
data = res.json()
```

API

뉴스

<https://newsapi.org/v2/top-headlines?country=kr&apiKey=9f5baf7d9f3f42879a20d7d19d9886e4>

<https://newsapi.org/v2/top-headlines?country=us&apiKey=9f5baf7d9f3f42879a20d7d19d9886e4>

날씨

<https://api.openweathermap.org/data/2.5/weather?lat=35.1728639953646&lon=129.130680529903>

[&units=metric&appid=6edee3c2aa182bc44d18ccb204c98a31](https://api.openweathermap.org/data/2.5/weather?lat=35.1728639953646&lon=129.130680529903&units=metric&appid=6edee3c2aa182bc44d18ccb204c98a31)

유튜브

<https://www.googleapis.com/youtube/v3/search?key=AlzaSyBcYq5zgED0rgC3aX00om0ThqbRPQ5V020&part=snippet&q=월드컵>

<http://ggoreb.com/api/youtube.jsp> [youtube2.jsp](#) [youtube3.jsp](#)

영화

https://api.themoviedb.org/3/search/movie?api_key=cba95d401a14ab806ffc13a5052aab89&query

=서울

연습문제) 거스름돈

당신은 음식점의 계산을 도와주는 점원입니다.

거스름 돈으로 사용할 수 있는 동전은 500원, 100원, 50원, 10원입니다.

손님에게 거슬러 주어야 할 돈이 N원일 때
거슬러 주어야 할 동전의 최소 개수를 구하세요.

n = 1260

count = 0

연습문제) 거스름돈 (여러번 빼기로 해결)

```
n = 1260
```

```
count = 0
```

```
while n > 0:
```

```
    if n >= 500: n -= 500
```

```
    elif n >= 100: n -= 100
```

```
    elif n >= 50: n -= 50
```

```
    elif n >= 10: n -= 10
```

```
    count += 1
```

```
print(count)
```

연습문제) 거스름돈 (나누기로 효율성 높임)

```
n = 1260
count = 0

coin = n // 500 # 동전 개수
count += coin
n %= 500

coin = n // 100 # 동전 개수
count += coin
n %= 100

coin = n // 50 # 동전 개수
count += coin
n %= 50

coin = n // 10 # 동전 개수
count += coin
n %= 10

print(count)
```

연습문제) 거스름돈 (최종 완성)

```
n = 1260
count = 0
for money in [500, 100, 50, 10]:
    coin = n // money
    count += coin
    n %= money
print(count)
```

7.23.(목)

연습문제) 1이 될 때까지

어떠한 수 N 에 1이 될 때까지 아래 두 과정 중 하나의 연산을 수행해야 됩니다.

1) N 에서 1을 뺍니다.

2) N 을 K 로 나눕니다.

단, 두번째 연산은 N 이 K 로 나누어 떨어질 때만 수행 됩니다.

ex) N 이 17, K 가 4 일때

1단계) $17 - 1 \rightarrow 16$ (나누어 떨어지지 않으므로)

2단계) $16 / 4 \rightarrow 4$ (나누어 떨어짐)

3단계) $4 / 4 \rightarrow 1$ (나누어 떨어짐)

1이 될 때까지 수행해야 하는 최소 횟수를 구하세요.

입력 예시) 17 4

출력 예시) 3

연습문제) 1이 될 때까지 (나머지를 구해서 한꺼번에 빼기)

N, K = 32, 11

count = 0

while N > 1:

if N % K == 0:

N //= K

count += 1

else:

뺄값 = N % K

N -= 뺄값

count += 뺄값

K가 N보다 작아지면 나머지 연산자에 의해 0이
되어버림

ex) N = 2 K = 11

N -= 2 → 0

print(f'현재 N: {N}, 횟수: {count}')

연습문제) 1이 될 때까지 (횟수가 1번 더 늘어나는 버그 수정)

N, K = 32, 11

count = 0

while N > 1:

if N % K == 0:

N //= K

count += 1

else:

N이 K보다 작아지면 K로 나눌 수 없으므로, (N - 1)만큼 한 번에 빼고 종료

if N < K:

count += (N - 1)

N = 1

print(f'현재 N: {N}, 횟수: {count}')

break

뺄값 = N % K

N -= 뺄값

count += 뺄값

print(f'현재 N: {N}, 횟수: {count}')

함수

어떠한 값을 입력하면 정의된 절차에 따라 결과물을 내는 것

1. 함수명
2. 반환값
3. 매개변수

함수는 자료형으로 사용 가능

1. 함수는 변수에 저장할 수 있음
2. 함수는 다른 함수의 매개변수로 전달될 수 있음
3. 함수가 실행되려면 소괄호 () 사용

그래프 출력하기

도구 설치 - `pip install matplotlib`

matplotlib의 pyplot 사용

```
import matplotlib.pyplot
```

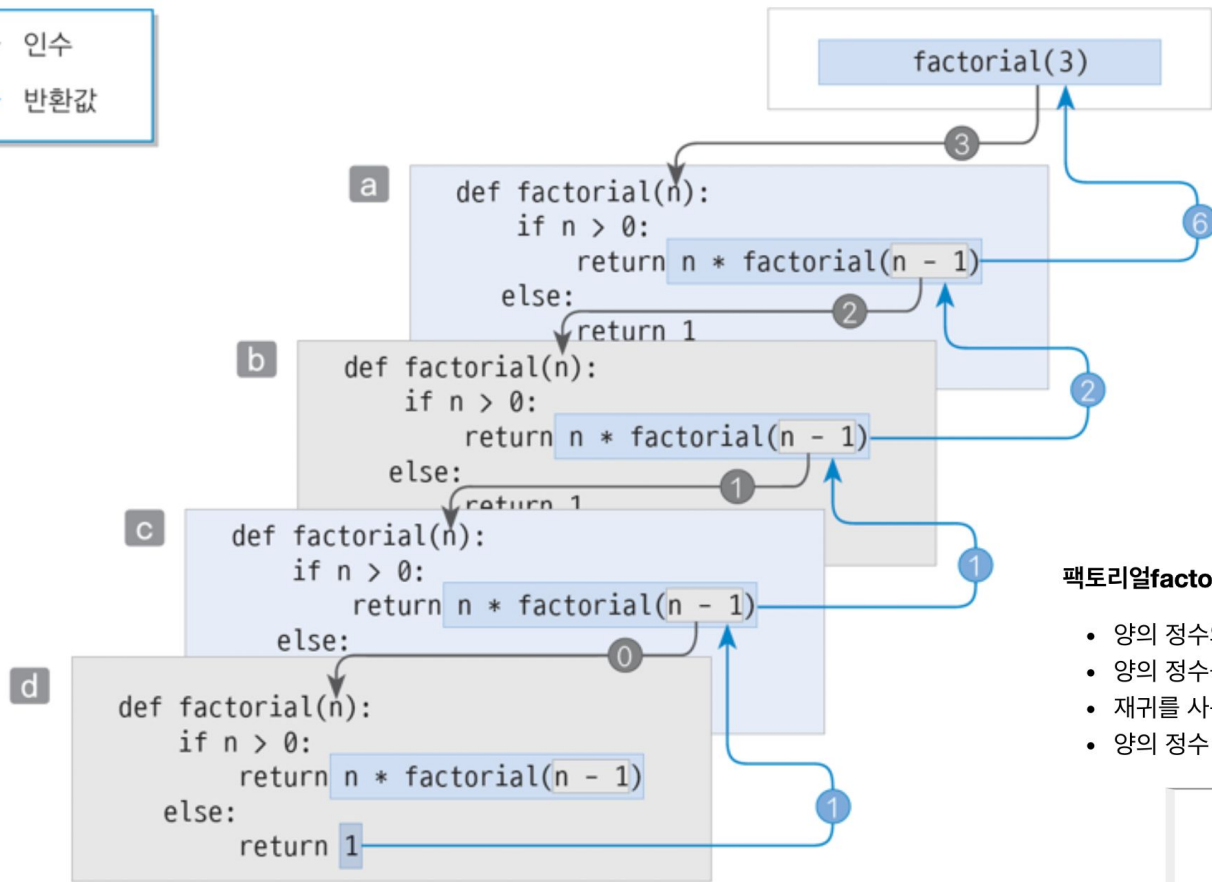
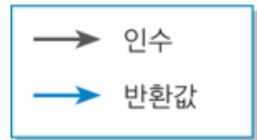
```
matplotlib.pyplot.plot([10, 1, 3, 5, 4], marker='*', color='red')
```

함수 문제 (1부터 n까지 합 구하기)

$$a_n = \sum_{k=1}^n \frac{k(k+1)}{2}$$

$$a_n = \frac{n(n+1)(n+2)}{6}$$

재귀 호출 recursive call



팩토리얼 factorial

- 양의 정수의 곱을 구하는 문제
- 양의 정수를 순서대로 곱한다는 의미로 순차 곱셈이라고도 함
- 재귀를 사용하는 대표적인 예
- 양의 정수 n 의 팩토리얼($n!$)의 재귀적 정의

- $0! = 1$
- $n > 0$ 이면 $n! = n \times (n - 1)!$

최대 공약수

GCD : Greatest Common Divisor

A, B 두 수 중 작은 수를 기준으로 1씩 감소시키면서

A와 B가 나누어 떨어지는 숫자

유클리드 호제법Euclidean algorithm

- x와 y가 있고 x를 y로 나눈 나머지 r이 있을 때
- x와 y의 최대 공약수는 y와 r의 최대 공약수와 같다

단계	A	B	R (나머지)
1	192	162	30
2	162	30	12
3	30	12	6
4	12	6	0

최대 공약수

최소 공배수

LCM : Least Common Multiple

A, B 두 수 중 큰 수를 기준으로 1씩 증가시키면서

A와 B가 나누어 떨어지는 숫자

유클리드 호제법 **Euclidean algorithm**

- x 와 y 가 있고 $x * y$ 를 최대 공약수로 나눈 몫은 최소 공배수이다

변수의 유효범위

○ 함수 내에서 선언된 변수는 지역변수

def 함수명(매개변수):

age = 10

 return 반환값

※ if, while, for 내에서 선언된 변수는 전역변수

Lambda

함수명 → 변수명

매개변수 → **lambda** 매개변수

반환값 → : 반환값

ex) minus = lambda a, b: a - b

람다 연습문제

```
def myfunc(numbers):  
    result = []  
    for number in numbers:  
        if number > 5:  
            result.append(number)  
    return result
```

```
myfunc2 = lambda numbers: [ n for n in numbers if n > 5 ]
```

```
result = myfunc2([2, 3, 4, 5, 6, 7, 8])
```

```
print(result)
```

코드를 간결하게 만드는 한줄 표현식들

1. if → 조건부 표현식
2. for → 리스트 컴프리헨션
3. 함수 → 람다식

파일 종류

1. 텍스트

메모장, 편집기로 내용을 확인할 수 있는 형태

- 인코딩 종류

utf-8 - 한글 등 특수문자 3Byte

euc-kr (cp949, ms949) - 한글 등 특수문자 2Byte

2. 바이너리

특수한 프로그램을 사용해서 내용을 확인할 수 있는 형태

서울시 기온 그래프 그리기

```
f = open('seoul_기온.csv', 'rt')
date_list = []
min_list = []
for data in f:
    #      0      1      2      3
    # 2021-01-02,-5,-8.4,-1.4
    date_list += [data.split(',')[0]]
    min_list.append(data.split(',')[2])
f.close()

date_list = [ date for date in date_list[1:] ]
min_list = [ float(min) for min in min_list[1:] ]

import matplotlib.pyplot as plt
plt.rcParams['axes.unicode_minus'] = False
plt.plot(min_list)
plt.xticks(range(len(min_list)), date_list, rotation=45)
plt.show()
```

Data Uri (base64 이미지)

```
file = 'c:/image.png'
```

```
import base64
```

```
file = open(file, 'rb')
```

```
base64_string = base64.b64encode(file.read())
```

```
file.close()
```

```
print('' % base64_string.decode())
```

텍스트 파일 내용 읽기

read() - 내용 전체

for 변수 in 파일객체: - 한줄씩

readline() - 한줄만 (\n 문자 기준)

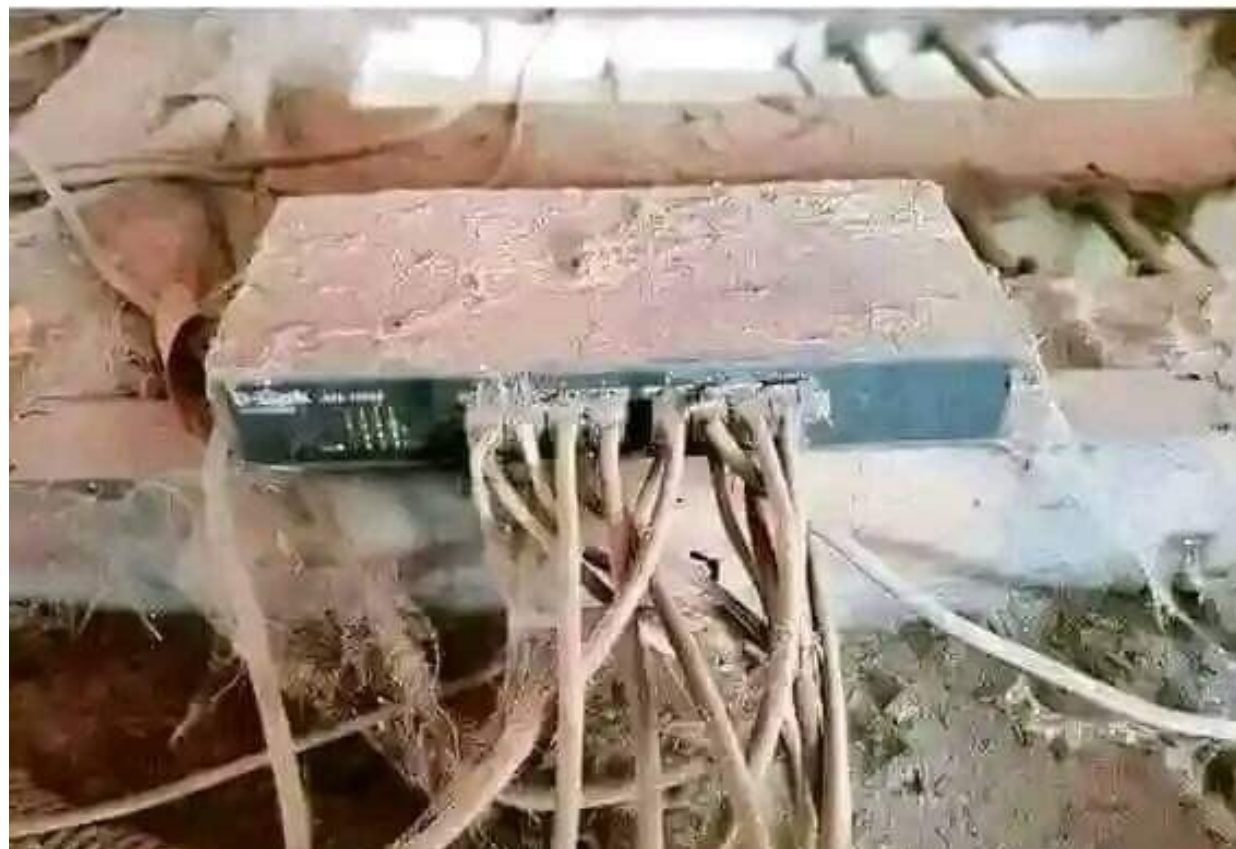
readlines() - 내용 전체 (리스트)

텍스트 파일 내용 쓰기

write() - 문자를 파일의 내용으로 쓰기

writelines() - 리스트를 파일의 내용으로 쓰기

If it works, don't touch it



파일쓰기 연습문제 (시스템 로그 파일 생성)

프로그램이 실행되면서 발생하는 가상의 기록(로그)을 `system.log` 파일에 텍스트 형식으로 누적 저장합니다.

조건 1) 기존 파일 내용을 지우지 않고 이어서 작성

조건 2) 각 로그는 줄바꿈하며 한 줄에 하나씩 작성

```
import datetime
```

```
# 현재 날짜와 시간을 구하기
```

```
current_time = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

```
# 기록할 가짜 로그 데이터
```

```
log_messages = [
```

```
    "사용자가 관리자 페이지에 로그인했습니다.",
```

```
    "데이터베이스 연결에 성공했습니다.",
```

```
    "경고: 디스크 용량이 80% 이상 채워졌습니다.",
```

```
    "에러: 파일 업로드 중 예기치 못한 오류가 발생했습니다."
```

```
]
```

```
print("로그 생성을 시작합니다...")
```

```
# [문제] 반복문을 사용하여 system.log 파일에 로그를 한 줄씩 누적해서 쓰기
```

```
print("로그 파일 작성 완료!")
```

파일쓰기 연습문제 (이미지 다운로드)

인터넷 주소(URL)에 있는 이미지 파일을 파이썬 코드로 다운로드하여 내 컴퓨터에 `python-image.jpg` 로 저장합니다.

```
import requests
```

```
# 다운로드할 파이썬 로고 이미지 주소입니다.
```

```
image_url = "http://ggoreb.com/python/images/python-image.jpg"
```

```
print("웹에서 이미지를 요청하는 중...")
```

```
# requests를 사용해 이미지 데이터를 가져옵니다.
```

```
response = requests.get(image_url)
```

```
image_bytes = response.content
```

```
print("이미지 다운로드 완료! 이제 파일로 저장합니다...")
```

```
# [문제] 받아온 이미지 바이너리 데이터(image_bytes)를 'python-image.jpg' 파일로 저장하기
```

```
print("이미지 파일 저장 완료! 폴더에서 이미지 파일을 확인해보세요.")
```

7.25.(금)

해시 Hash

- 해시(Hash)

임의의 길이 데이터를 고정된 길이의 (거의) 고유한 값으로 변경하는 것

- 해시값 / 해시코드 (Hash Code)

일반적으로 해시값 또는 해시코드 라고 부름

- 해시 함수 (Hash Function)

해시를 생성해주는 함수

* **SHA-256**의 경우 **2**의 **256**승을 의미하고

전세계 **80**억 인구가 **1**초에 하나씩 **136**해 년 동안 시도해야 맞출 정도의 확률
결론은 중복될 확률 **0%**

문자열 해시 (비밀번호 암호화)

```
import hashlib
```

```
old_pwd = '1234'
```

```
m = hashlib.sha256()
```

```
m.update(old_pwd.encode())
```

```
new_pwd = m.hexdigest()
```

```
# new_pwd = hashlib.sha256(old_pwd.encode()).hexdigest()
```

```
print(new_pwd)
```

정규 표현식으로 할 수 있는 행위

1. 검사

- 문자열 찾아내기
- `find (search())`

2. 추출

- 문자열 잘라내기
- `slicing (findall())`

3. 치환

- 문자열 변경하기
- `replace (sub())`

정규 표현식 기본 코드

```
import re
```

```
규칙 = re.compile( '[^가-힣]' )
```

```
문장 = '가나라abcd가나라라다'
```

```
결과 = 규칙.search( 문장 )
```

```
if 결과: print('통과')
```

```
else: print('불가')
```

1. re 도구 불러오기
2. pattern 지정 - compile()
3. 확인 - search()
 - 1개 확인: search() N개 확인: findall()

정규 표현식 사용 기호

[] 검사할 문자

{ } 등장 횟수

. 모든 문자

* 등장 횟수 (0번 이상)

+ 등장 횟수 (1번 이상)

^ 시작

\$ 종료

\ 특수문자

() 그룹

? 등장 횟수 (0 ~ 1번, Non-Greedy)

정규 표현식

== 단독 사용이 가능한 기호 ==

[] 대괄호

지정된 문자에 “또는” 개념 적용

범위표현 - (하이픈)

[^] 부정

\d 숫자 **\s** 공백 **\w** 문자

. 모든문자 (줄바꿈 빼고)

단, 대괄호 안의 .은 실제 마침표

정규 표현식

== 단독 사용 불가, 위의 기호와 같이 사용해야 하는 기호==

* 0개 ~ N개

+ 1개 ~ N개

{ } {숫자} 숫자만큼

{숫자1,숫자2} 숫자1~숫자2

? 0개 ~ 1개

greedy스러운 기호를 non-greedy 변경

^ 시작 \$ 종료

| 여러개의 패턴 (or)

() 그룹화

한글, 일본어, 한자 범위 표현

```
pattern = re.compile(r'[가-힣ぁ-んァ-ンㄱ-ㅎ\u4E00-\u9FFF]+')
```

언어/문자 종류	코드 범위 (16진수)	정규식 표현	예시 문자	비고
한글	AC00-D7A3	[가-힣]	가, 나, 다	완성형 한글 (11172자)
히라가나	3040-309F	[ぁ-ん]	あ, い, う	일본어 고유 문자
가타카나	30A0-30FF	[ァ-ン]	ア, イ, ウ	외래어·이름 표기
반각 가타카나	FF65-FF9F	[ㄱ-ㅎ]	ア, ハ, シ	옛 Shift-JIS 호환용
한자	4E00-9FFF	[\u4E00-\u9FFF]	中, 國, 漢	현대 중국어 한자 대부분
한자 확장 A	3400-4DBF	[\u3400-\u4DBF]	ㄱ, ㄴ	드물게 쓰이는 한자
(선택) 확장 B~F	20000-2EBEF	[\u20000-\u2EBEF]	𠂇 등	고대 문자·희귀 한자 포함

7.27.(월)

로또 번호

<http://ggoreb.com/python/html/number.html>

```
import requests
res = requests.get('http://ggoreb.com/python/html/number.html')
res.encoding = None
text = res.text
```

```
import re
pattern = re.compile(r'ball[0-9]+>([0-9]+)')
res = pattern.findall(text)
print(len(res))
print(res)
```

```
<span class="ball_645_lrg ball1">2</span>
```

숨어있는 문자 찾기

문서 : <http://ggoreb.com/quiz/special.html>[illegible]

```
import requests as req
```

```
res = req.get('http://ggoreb.com/quiz/special.html')
```

```
res.encoding = None
```

```
text = res.text
```

```
print(text)
```

숨어있는 문자 찾기

```
import requests  
res = requests.get('http://ggoreb.com/quiz/special.html')  
res.encoding = None  
text = res.text  
idx = text.find('%%$')  
text = text[idx:]
```

```
import re  
pattern = re.compile(r'[a-z]+')  
res = pattern.findall(text)  
print("".join(res))  
print(len(res))  
print(res)
```

동일한 패턴 문자 찾기

문서 : <http://ggoreb.com/quiz/pattern.html>

- 조건에 맞는 문자 찾기 (소문자 10개)

```
<!--
find rare characters in the mess below:
-->

<!--
kAewtlOYgcFQaJNhHVgXxDiQmzjfcPYbzxIWrvCqsmUbCunkfxZWDZjUZMiGqhRRiUvGmYmVnJIHEmbT
MUKLECKdCthezSYBpIEIRnZugFAxDRTQPpyeCBgBfaRVvvguRXLvkAdLOeCKxsDUvBBCwdpMMWmuELEg
ENihrpCLhujoBqPRDPvfzCwadMMmbkmkzCCzoTPfbrIzBqMbImxTxNniNoCufprWXxgHZpIdkoLCrHJq
vYuyJFCZtqXLhWiYzOXeglkzhVJIWmeUySGuFvmlTCyMshQtvZpPwulbOHNoBauwvuJYcmqznOBgByPw
TssUkHtTACKNubWUeFgZtZtHhTChUcTHLhYVZQgJLYLH5FmNBY
```

조건 : 양쪽에 대문자 **3개로만** 둘러싸여 있는 **소문자**

ex) aUSEhABDa (O), xxTYNiPERu (O), **EHGRiDKJa** (X), **HGRiXCRZ** (X)

정규표현식에서 자주 사용되는 플래그 옵션

플래그	축약 형	주요 역할	추천 활용 상황
re.IGNORECASE	re.I	대소문자 구분 없음	영문 텍스트 검색, 회원가입 ID/이메일 검증
re.MULTILINE	re.M	다중 행 모드 (^, \$가 줄바꿈 기준으로 작동)	여러 줄로 구성된 텍스트의 각 줄 첫/끝 단어 찾기
re.DOTALL	re.S	. 이 줄바꿈(\n)도 포함	HTML/JSON 태그 내 여러 줄 텍스트 긁어오기
re.ASCII	re.A	\w, \b 등을 ASCII 범위(영문/숫자)로 제한	한글/유니코드 제외하고 순수 영문·숫자만 뽑기

매장 데이터

<http://ggoreb.com/python/file/shop.txt>

○ 데이터 구성 형식

- 1) 매장명: 숫자 2자리 / 영어, 숫자, 한글, 띄어쓰기
- 2) 매장설명
- 3) 메뉴 및 가격
- 4) 주소 / 전화번호 / 영업시간 / 휴무일

○ 전체 매장수: 564개

매장 데이터

문제 1) 모든 매장명 추출하기

각 매장 이름이 적힌 줄(예: 01 무시로, 02 삼형제오리)만 모두 추출하세요.

총 매장 수는 **564개** 입니다.

```
import re, requests
res = requests.get('http://ggoreb.com/python/file/shop.txt')
res.encoding = 'utf8'
text = res.text
p = re.compile(r'코드 작성', re.M)
result = p.findall(text)
print(len(result))
print(result[:3])
```

[참고사항]

- 1) 매장명 줄은 항상 두 자리 숫자와 공백으로 시작합니다.
- 2) 매장명에 한글, 영어, 숫자, 띄어쓰기가 섞여있을 수 있습니다.

매장 데이터

문제 2) 지역(구/군) 이름 추출하기

꺾쇠괄호로 표시된 지역(예: <영도구> <서구>)을 모두 추출하세요.

총 지역 수는 **15개** 입니다.

```
import re, requests
res = requests.get('http://ggoreb.com/python/file/shop.txt')
res.encoding = 'utf8'
text = res.text
p = re.compile(r'코드 작성', re.M)
result = p.findall(text)
print(len(result))
print(result)
```

[참고사항]

1) <영도구>처럼 괄호 다음 바로 문자가 나오는
경우와

<서구>처럼 공백이 끼여있는 경우가 있습니다.

매장 데이터

문제 3) 주소 추출하기

정보 줄 (**부산시 ...** / 전화번호 / 영업시간 / 휴무일) 에서 **주소 부분만** 잘라내세요.
(전화번호, 영업시간, 휴무일은 제외하고 **"부산시"부터 첫 번째 / 전까지만**)

```
import re, requests
res = requests.get('http://ggoreb.com/python/file/shop.txt')
res.encoding = 'utf8'
text = res.text
p = re.compile(r'코드 작성', re.M)
result = p.findall(text)
print(len(result))
print(result[:3])
```

[참고사항]

1) .+ 표현은 greedy 해서 마지막 / 까지 다 가져옵니다.

2) ? 를 붙여서 .+? 처럼 쓰면 non-greedy로 바뀌어서

첫번째로 등장하는 / 에서 멈출 수 있습니다.

매장 데이터

문제 4) 전화번호 추출하기

정보 줄 (부산시 ... / **전화번호** / 영업시간 / 휴무일) 에서 **전화번호 부분만** 잘라내세요.
(전화번호에 대한 내용이 빠진 매장이 6개 있습니다)

```
import re, requests
res = requests.get('http://ggoreb.com/python/file/shop.txt')
res.encoding = 'utf8'
text = res.text
p = re.compile(r'코드 작성', re.M)
result = p.findall(text)
print(len(result))
print(result[:3])
```

[참고사항]

- 1) .+ 표현은 **greedy** 해서 마지막 / 까지 다 가져옵니다.
- 2) ? 를 붙여서 .+? 처럼 쓰면 **non-greedy**로 바뀌어서
첫번째로 등장하는 / 에서 멈출 수 있습니다.
- 3) "전화불가", "전화문의 불가", "1688-1688" 과 같이
전화번호의 형식이 다른 매장도 있습니다.

매장 데이터

문제 5) 메뉴 추출하기

각 매장의 **모든 메뉴**를 추출하세요.

```
import re, requests
res = requests.get('http://ggoreb.com/python/file/shop.txt')
res.encoding = 'utf8'
text = res.text
p = re.compile(r'코드 작성', re.M)
result = p.findall(text)
print(len(result))
print(result[:3])
```

[참고사항]

- 1) | (파이프) 기호는 정규표현식에서
특수 기호 (or, 여러 패턴 중 하나)로 쓰이기 때문에
문자 그대로의 | 를 찾으려면
이스케이프 문자 \ 와 함께 \|로 써야 합니다.
- 2) .+ 는 **greedy** 이기 때문에
한 줄에 | 가 여러 번 나오면 마지막 | 까지
욕심내서
다 가져가 버립니다.

종합 연습문제 (1장 ~ 6장)

문제 1) 회식비 정산하기 (수학/계산 내장함수)

팀 회식을 마치고 총 결제금액을 인원수대로 나누려고 합니다.

1인당 내야 하는 금액과 나머지 금액 구하세요.

또한 100원 단위로만 정산이 가능한 경우를 대비해 반올림 처리를 해주세요.

```
total_price = 187500
```

```
people = 7
```

```
# 코드 작성 (divmod 사용)
```

```
print(f'1인당 {share}원, 나머지 {remain}원은 총무가 부담')
```

```
# 내야 하는 반올림해서 보정한 1인당 금액 (rounded)
```

```
# 코드 작성 (round 사용)
```

```
print(f'반올림 정산 시 1인당 {rounded}원')
```

[참고사항]

1) divmod()를 사용하면 몫과 나머지를 한 번에 구할 수 있습니다.

2) round()의 두 번째 인자로 반올림 자릿수를 지정할 수 있습니다.

[실행결과]

1인당 26785원, 나머지 5원은 총무가 부담
반올림 정산 시 1인당 26800.0원

종합 연습문제 (1장 ~ 6장)

문제 2) 이번 달 매출 순위표 만들기 (순서/반복 내장함수)
영업팀 직원별 이번 달 매출 데이터가 있습니다.
매출이 높은 순서대로 순위를 매겨 출력하세요.

```
names = ['김민준', '이서연', '박도윤', '최하은', '정지호']  
sales = [3200000, 4550000, 2980000, 4550000, 3870000]
```

```
# 이름과 매출을 하나로 묶고, 매출 내림차순으로 정렬한 뒤  
# 순위(1위부터)와 함께 출력하세요  
# 코드 작성 (zip, sorted, enumerate 사용)
```

[참고사항]

1) sorted()의 key와 reverse 옵션을 활용해보세요.

사용 예1) 딕셔너리 정렬

```
items1 = [{'name': 'a'}, {'name': 'c'}, {'name': 'b'}]  
sorted_items1 = \  
    sorted(items1, key=lambda x: x['name'],  
           reverse=True)  
print(sorted_items1)
```

사용 예2) 튜플 정렬

```
items2 = [('김', 20), ('이', 30), ('박', 25)]  
sorted_items2 = sorted(items2, key=lambda x: x[0])  
print(sorted_items2)
```

[실행결과]

0위 이서연 4550000원

1위 최하은 4550000원

...

종합 연습문제 (1장 ~ 6장)

문제 3) 회원가입 폼 유효성 검사하기 (논리/검사 내장함수)

회원가입 시 입력받은 정보가 조건을 만족하는지 검사하는 함수를 만드세요.

```
def check_signup(user_id, password, age):  
    # 조건 1) user_id는 문자열이어야 함  
    # 조건 2) password 길이는 8자 이상이어야 함  
    # 조건 3) age는 정수형이며 14세 이상이어야 함  
    # 세 조건을 모두 만족해야 True, 하나라도 어기면 False  
    # 코드 작성 (isinstance, all 사용)  
    pass
```

```
print(check_signup('minjun01', 'pw12345678', 20)) # True  
print(check_signup('minjun01', 'pw123', 20))        # False (비밀번호 짧음)  
print(check_signup('minjun01', 'pw12345678', '20')) # False (age가 문자열)
```

[참고사항]

- 1) `isinstance()`(값, 타입)으로 자료형을 검사할 수 있습니다.
- 2) 여러 조건을 리스트로 만들고 `all()`로 한 번에 검사해보세요.

[실행결과]

True
False
False

종합 연습문제 (1장 ~ 6장)

문제 4) 이벤트 쿠폰 코드 만들기 (문자/코드 내장함수)

이벤트 당첨자에게 발급할 쿠폰 코드를 만들려고 합니다.

회원 아이디의 각 글자를 유니코드 숫자로 바꾼 뒤

그 합을 16진수로 표현해서 쿠폰 코드 뒷자리로 사용합니다.

```
user_id = 'minjun01'
```

```
# 1) 각 글자를 유니코드 코드값으로 변환해서 리스트로 만들기
```

```
# 코드 작성 (ord 사용)
```

```
# 2) 코드값의 합을 구하고, 16진수 문자열로 변환하기
```

```
# 코드 작성 (sum, hex 사용)
```

```
print(f'쿠폰코드: EVENT-{hex_code}')
```

[참고사항]

1) `ord()`는 문자 하나를 정수(유니코드 코드값)로 바꿔줍니다.

2) `hex()`의 결과에는 앞에 `0x`가 붙으니 슬라이싱으로 제거하세요.

[실행결과]

쿠폰코드: EVENT-2f2

종합 연습문제 (1장 ~ 6장)

문제 5) 재고 데이터 일괄 처리하기 (기타/응용 내장함수)

창고 재고 시스템에서 받아온

문자열 형태의 재고 수량 데이터를 정리하려고 합니다.

```
stock_raw = ['120', '35', '8', '250', '64']
```

1) 문자열 리스트를 정수 리스트로 변환한 뒤

재고가 50개 미만인 것만 추려내기

코드 작성 (map, filter, lambda 사용)

```
print('재고부족 목록:', low_stock)
```

2) 재고가 100개 이상인 상품은

창고 정리를 위해 절반만 남기고 나머지는

타 지점으로 이관을 위해 리스트를 만드세요. 120 → 60, 250 → 125

코드 작성 (map, lambda 사용)

```
print('이관 수량:', transfer_list)
```

[참고사항]

1) map(함수, 리스트)는 리스트의 모든 요소에 함수를 적용합니다.

2) filter(함수, 리스트)는 조건을 만족하는 요소만 남깁니다.

3) 소수점이 나오지 않도록 정수 나눗셈(//)을 사용하세요.

[실행결과]

재고부족 목록: [35, 8]

이관 수량: [60, 125]

종합 연습문제 (1장 ~ 6장)

문제 6) 거래처 API 응답 데이터 파싱하기 (json)

거래처 시스템에서 아래와 같은 JSON 형태의 주문 데이터를 받았습니다.
배송 대기 중인 주문의 고객명만 추려내세요.

```
import json
response = '''
[
  {"order_id": 101, "customer": "김민준", "status": "배송중"},
  {"order_id": 102, "customer": "이서연", "status": "배송대기"},
  {"order_id": 103, "customer": "박도윤", "status": "배송대기"},
  {"order_id": 104, "customer": "최하은", "status": "배송완료"}
]
```

1) JSON 문자열을 파이썬 객체로 변환하세요

코드 작성

2) status가 '배송대기'인 주문의 customer만 리스트로 뽑으세요

코드 작성

print(waiting_customers)

[참고사항]

1) json.loads()는 JSON 문자열을
리스트/딕셔너리로 변환해줍니다.

[실행결과]

['이서연', '박도윤']

종합 연습문제 (1장 ~ 6장)

문제 7) 경품 이벤트 당첨자 추천하기 (random)

인스타그램 팔로우 이벤트 참여자 명단 중에서 랜덤으로 당첨자를 뽑는 프로그램을 만드세요.

```
import random
```

```
participants = [  
    'user_a', 'user_b', 'user_c', 'user_d', 'user_e',  
    'user_f', 'user_g', 'user_h', 'user_i', 'user_j'  
]
```

1) 참여자 명단을 무작위로 섞은 후 출력하세요
코드 작성 (random.shuffle 사용)

2) 1등 1명, 2등 2명을 중복 없이 뽑으세요
코드 작성 (random.sample 사용)

```
print('1등:', first_prize)  
print('2등:', second_prize)
```

[참고사항]

1) random.shuffle()은

리스트 자체를 무작위로 섞습니다 (반환값 없음)

2) random.sample(리스트, n)은

중복 없이 n개를 뽑습니다.

[실행결과]

참여자 명단: ['user_j', 'user_g', ... 'user_b']

1등: user_g

2등: ['user_d', 'user_c']

종합 연습문제 (1장 ~ 6장)

문제 8) 서버 백업 파일명 자동 생성 & 실행시간 측정 (time, datetime)

서버 백업을 실행할 때마다 현재 시각이 담긴 파일명을 자동으로 만들고 백업 작업(예: 파일 압축)에 걸린 시간을 측정하려고 합니다.

```
import time
import datetime

# 1) 'backup_20260726_153045.zip' 같이
#   현재 일시를 이용해 파일명을 만드세요.
# 코드 작성 (datetime.datetime.now(), strftime 사용)
print(backup_filename)

# 2) 아래 작업에 소요된 시간을 측정하세요
start = time.time()
total = 0
for i in range(30_000_000):
    total += i
# 코드 작성 (걸린 시간 = 현재시각 - start)
print(f'백업 소요시간: {elapsed:.2f}초')
```

[참고사항]

- 1) strftime('%Y%m%d_%H%M%S') 형식으로 문자열을 만들 수 있습니다.
- 2) time.time()은 실행 시점의 유닉스 타임스탬프를 반환합니다.

[실행결과]

backup_20260726_105920.zip
백업 소요시간: 1.91초

이메일 발송 코드 - [email.ipynb](https://github.com/Amugona/email.ipynb)

```
import shutil
import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
```

Gmail 계정 정보 (앱 비밀번호 사용!)

```
SMTP_SERVER = "smtp.gmail.com"
SMTP_PORT = 587
SENDER = "amugona.2606@gmail.com"
PASSWORD = "orzkgmnwuhmubgqp"
RECIPIENT = "seorab@naver.com"
```

여러명에게 발송할 경우 리스트로 지정

```
# RECIPIENT = ["seorab@naver.com", "seorab81@gmail.com"]
```

1. 디스크 사용량 확인

```
def check_disk_usage(path="/", threshold=80):
    total, used, free = shutil.disk_usage(path)
    usage = used / total * 100
    return usage, threshold
```

2. 이메일 발송 함수

```
def send_email(subject, body):
    msg = MIMEMultipart()
    msg["From"] = SENDER
    msg["To"] = RECIPIENT
```

```
# 여러명에게 발송할 경우 구분자를 넣어 문자열로 변환
# msg["To"] = ', '.join(RECIPIENT)
```

```
msg["Subject"] = subject
msg.attach(MIMEText(body, "html"))
```

try:

```
with smtplib.SMTP(SMTP_SERVER, SMTP_PORT) as server:
    server.starttls() # TLS 암호화 연결
    server.login(SENDER, PASSWORD) # 로그인
    server.send_message(msg) # 메일 발송
```

except Exception as e:

```
print(f"❌ 발송 실패 오류 발생: {e}")
```

3. 메인 로직

```
if __name__ == "__main__":
```

```
    usage, threshold = check_disk_usage("/")
```

```
    subject = f"[경고] 디스크 사용량 {usage:.1f}% 초과"
```

```
    body = f"""
```

```
<h3>⚠️ 디스크 경고</h3>
```

```
<p>현재 디스크 사용량은 <b>{usage:.1f}%</b> 입니다.<br>
```

```
서버 점검이 필요합니다.</p>
```

```
"""
```

```
    send_email(subject, body)
```

```
    print(f"⚠️ 이메일 발송 완료")
```

이메일 발송

☆ [경고] 디스크 사용량 77.0% 초과

^ 보낸사람

amugona.2606@gmail.com

받는사람

seorab@naver.com

seorab81@gmail.com

2026년 7월 26일 (일) 오후 12:32

! 디스크 경고

현재 디스크 사용량은 77.0% 입니다.
서버 점검이 필요합니다.

[경고] 디스크 사용량 77.0% 초과 > 받은편지함 x



amugona.2606@gmail.com

seorab, 나에게 ▾

! 디스크 경고

현재 디스크 사용량은 77.0% 입니다.
서버 점검이 필요합니다.

구글 이메일 앱비밀번호 발급

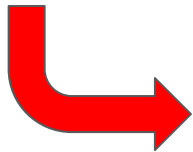
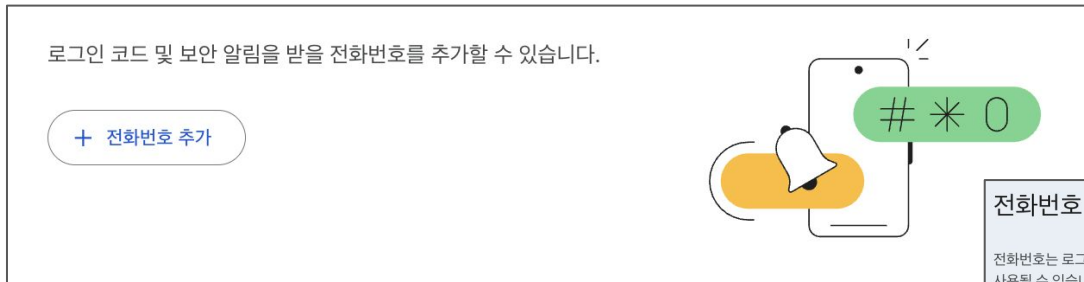
1. 구글 gmail.com 계정 준비 (ooo@gmail.com)
2. [Google 계정 관리] → [보안 및 로그인]
→ [2단계 인증]



패스키, 전화번호 등 2단계 인증 수단
추가

구글 이메일 앱비밀번호 발급

1. 구글 gmail.com 계정 준비 (ooo@gmail.com)
2. [Google 계정 관리] → [보안 및 로그인]
→ [2단계 인증 전화번호]



전화번호 추가

전화번호는 로그인 시 본인 인증에 사용되며 비정상적인 활동이 감지된 경우 알림을 받기 위해 사용될 수 있습니다.

 ▼ +8201084788182

Google Voice 번호를 사용할 수는 있지만, Google 계정에 액세스할 수 없게 된 경우 이 번호로 코드를 받을 수 없습니다. 이동통신사 요금이 부과될 수 있습니다. [Google이 정보를 사용하는 방법 자세히 알아보기](#) ⓘ

☒ 문자 메시지로 코드 받기

☐ 음성 메시지로 코드 받기

구글 이메일 앱비밀번호 발급

1. 구글 gmail.com 계정 준비 (ooo@gmail.com)
2. [Google 계정 관리] → [보안 및 로그인]
→ [2단계 인증 사용 설정]



2단계 인증 사용 설정
로그인 시 2단계로 전화번호를 사용하여 계정 보안을 한층 강화하세요.

사용 설정

다음 번호로 로그인 코드를 받을 수 있습니다. Google 계정 복구에 사용할 수 있는 다른 번호가 더 있을 수 있습니다. [복구 전화번호 관리](#)

 010-8478-8182



구글 이메일 앱비밀번호 발급

1. 구글 gmail.com 계정 준비 (ooo@gmail.com)
2. [Google 계정 관리] → [보안 및 로그인]
→ [2단계 인증] → [2단계 인증 사용 설정]

2단계

Google 계정에 액세스할 수 있도록 이 정보를 최신 상태로 유지하고 로그인 옵션을 추가하세요.

	패스키 및 보안 키	 패스키 1개
	Google 메시지	 기기 1대
	OTP	 OTP 앱 추가
	전화번호	 010-8478-8182

2단계 인증 사용 설정

이제 내 계정이 2단계 인증으로 보호됩니다



로그인할 때 가장 안전한 2단계를 완료하라는 메시지가 표시되므로 이 정보가 항상 최신 상태여야 합니다.

	Google 메시지	 기기 1대
	전화번호	 010-8478-8182

구글 이메일 앱비밀번호 발급

3. Google 계정 검색에서 "앱 비밀번호" 검색

또는 <https://myaccount.google.com/apppasswords> 접속

→ "앱 이름" 입력 → 16자리 비밀번호 발급

← 앱 비밀번호

앱 비밀번호를 사용하면 최신 보안 표준을 지원하지 않는 오래된 앱 및 서비스에서 Google 계정에 로그인할 수 있습니다.

앱 비밀번호는 최신 보안 표준을 사용하는 최신 앱 및 서비스보다 보안 수준이 낮습니다. 앱 비밀번호를 만들기 전에 앱에 로그인하려면 비밀번호가 필요한지 확인해야 합니다.

[자세히 알아보기](#)

앱 비밀번호가 없습니다.

앱 전용 비밀번호를 새로 만들려면 아래에 앱 이름을 입력하세요...

앱 이름

생성된 앱 비밀번호

기기용 앱 비밀번호

orzkg gmnw uhmu bgqp

사용 방법

설정하려는 애플리케이션 또는 기기의 Google 계정 설정으로 이동합니다. 비밀번호를 위에 표시된 16자리 비밀번호로 교체합니다.

일반적인 비밀번호와 마찬가지로 이 앱 비밀번호는 Google 계정에 대한 완전한 액세스 권한을 부여합니다. 비밀번호를 기억하지 않아도 되므로 적어 놓거나 다른 사용자와 공유하지 마세요.

API 서버 헬스체크 모니터링 연습문제

문제) 회사에서 운영 중인 여러 **API** 서버가 있습니다.

매일 아침 이 서버들이 잘 살아있는지 자동으로 점검하고
문제가 있는 서버가 하나라도 있으면 담당자에게 메일로 알려주는 프로그램을
완성하세요.

모두 정상이면 메일은 보내지 않고 콘솔에만 정상 메시지를 출력합니다.

[health_check_email.ipynb](#)

[참고사항]

`requests.get(url)`은 서버가 500 에러를 응답하더라도
오류가 발생되지 않고, `res.status_code`에 500이
담긴

응답 객체를 돌려줍니다.

[실행결과]

⚠ 이메일 발송 완료 (1개 서버 이상)

☆ [경고] API 헬스체크 - 1개 서버 이상 감지 📧

보낸사람 amugona.2606@gmail.com

받는사람 seorab@naver.com

2026년 7월 26일 (일) 오후 12:57

⚠ API 헬스체크 결과

총 5개 중 1개 서버에서 문제가 발견되었습니다.

- 로또 API - 상태코드 500 (응답시간 0.22초)

7.28.(화)

모듈module

1. 한 개의 파이썬 파일이 하나의 모듈이 됨
2. 변수, 함수, 클래스를 가질 수 있음 (보통 함수)
3. 파이썬을 설치할 때 기본 표준 라이브러리들을 제공해주지만
필요하다면 "pip install" 을 통해 외부 모듈을 설치하는 것도 가능
ex) pip install requests
4. 모듈을 가져오는 2가지 방법
 - 1) import 모듈명
 - 2) from 모듈명 import 클래스/함수

예외 처리

1. `except` 사용 시 메시지 확인하기 → `Exception as e`
2. 오류의 종류를 정확히 지정하기 → `OOOError as e`
3. 외부 모듈을 사용중이라면 “외부 모듈.OOOError”
4. (오류발생) 코드의 범위 확인하기
→ `try: except:` 위치 선정

* 항상 사용자를 배려해서 후속조치를 할 수 있도록 코드 작성

HTTP 메소드 (서버와의 통신 방식)

1. 9가지 종류 (실제로는 2가지를 주로 사용)

POST, **GET**, PUT, DELETE

PATCH, HEAD, OPTIONS, CONNECT, TRACE

2. GET: ? 기호로 요청 데이터를 전달

POST: HTTP Body로 요청 데이터를 전달

GET은 requests.get(url, **params={...}**)

POST는 requests.post(url, **data={...}**) 또는 **json={...}**) 사용

동행복권 사이트에서 저번주 로또 1등 번호 확인하기

- 1) 목적 웹페이지로 접속
- 2) 개발자도구 열기
- 3) Network 탭에서 통신 내역 확인
- 4) GET/POST, Payload 등 요청 방식과 데이터 확인
- 5) 응답결과값 확인 - Text 인지 / JSON 인지
- 6) 데이터 파싱Parsing

동행복권 사이트에서 저번주 로또 1등 번호 확인하기

```
url = 'https://www.dhlottery.co.kr/lt645/selectPstLt645InfoNew.do'
```

```
query_params = {  
    'srchDir': 'center',  
    'srchLtEpsd': 1163,  
    '_': 1785206173568  
}
```

```
res = requests.get(url, params=query_params)
```

```
result = res.json()
```

```
num1 = result['data']['list'][0]['tm1WnNo']
```

```
num2 = result['data']['list'][0]['tm2WnNo']
```

```
num3 = result['data']['list'][0]['tm3WnNo']
```

```
num4 = result['data']['list'][0]['tm4WnNo']
```

```
num5 = result['data']['list'][0]['tm5WnNo']
```

```
num6 = result['data']['list'][0]['tm6WnNo']
```

```
print(num1, num2, num3, num4, num5, num6)
```

연습문제 - 다음 금융의 환율 정보 수집하기

회사에서 매일 아침 환율을 자동으로 확인하는 스크립트를 만들려고 합니다.

다음 금융 환율 페이지에 접속하면 화면에 실시간 환율이 표시되는데

이 데이터는 아래 **API**를 호출해서 받아옵니다.

그런데 이 **API**를 그냥 호출하면 응답을 받을 수 없습니다.

어떻게 하면 뚫을 수 있는지 요청헤더 값을 추가/수정하면서 찾아야 됩니다.

```
import requests
url = 'https://finance.daum.net/api/exchanges/summaries'
req_headers = {
    '???': '???'
}
res = requests.get(url, headers=req_headers)
print(res.status_code)
print(res.json())
```